02. Torrust Tracker 애플리케이션 초기화 과정 분석

2.1 메인 함수 분석 (`main()`)

함수명: `main()`

위치: `src/main.rs`

시그니처:

```
#[tokio::main]

async fn main() {

let (_app_container, jobs) = app::run().await;



tokio::select! {

_ = tokio::signal::ctrl_c() => {

tracing::info!("Torrust tracker shutting down ...");



jobs.cancel();



jobs.wait_for_all(Duration::from_secs(10)).await;



tracing::info!("Torrust tracker successfully shutdown.");
```

```
}

}

}
```

역할: 애플리케이션의 진입점으로서 전체 시스템을 시작하고 종료 신호를 처리합니다.

동작 과정:

1. **비동기 런타임 초기화**: `#[tokio::main]` 매크로를 통해 Tokio 비동기 런타임을 설정
2. **애플리케이션 시작**: `app::run().await` 를 호출하여 전체 애플리케이션과 작업 관리자를 초기화
3. **신호 대기**: `tokio::select!` 를 사용하여 CTRL+C 신호를 대기
4. **우아한 종료**: 종료 신호 수신 시 모든 작업을 취소하고 10초의 유예 시간을 두어 정리 작업 완료

관련 함수들:

- `app::run()` : 애플리케이션 전체 초기화 및 시작
- `JobManager::cancel()` : 모든 백그라운드 작업 취소 신호 전송
- `JobManager::wait_for_all()` : 모든 작업의 완료 대기

2.2 애플리케이션 시작 (`app::run()`)

함수명: `app::run()`

위치: `src/app.rs`

시그니처:

```rust
pub async fn run() -> (Arc<AppContainer>, JobManager) {

let (config, app_container) = bootstrap::app::setup();
```

```
let app_container = Arc::new(app_container);


let jobs = start(&config, &app_container).await;


(app_container, jobs)

}
```

역할: 애플리케이션의 전체 초기화 과정을 조율하고 실행 준비를 완료합니다.

동작 과정:

1. **부트스트랩 설정**: `bootstrap::app::setup()` 을 호출하여 설정과 컨테이너 초기화
2. **컨테이너 공유화**: `Arc::new()` 를 통해 애플리케이션 컨테이너를 스레드 안전하게 공유 가능하도록 래핑
3. **서비스 시작**: `start()` 함수를 호출하여 모든 서비스와 작업들을 시작
4. **반환**: 초기화된 컨테이너와 작업 관리자를 반환

관련 함수들:

- `bootstrap::app::setup()` : 설정 로딩 및 기본 컨테이너 초기화
- `start()` : 실제 서비스들의 시작 과정 관리

함수명: `app::start()`

위치: `src/app.rs`

시그니처:

```
#[instrument(skip(config, app_container))]

pub async fn start(config: &Configuration, app_container: &Arc<AppContainer>)
```

```
warn_if_no_services_enabled(config);


load_data_from_database(config, app_container).await;


start_jobs(config, app_container).await
}
```

역할: 설정 검증, 데이터 로딩, 그리고 모든 서비스 작업들의 시작을 순차적으로 수행합니다.

동작 과정:

1. **설정 검증**: `warn_if_no_services_enabled()` 로 활성화된 서비스가 있는지 확인
2. **데이터 로딩**: `load_data_from_database()` 로 데이터베이스에서 필요한 데이터 로드
3. **작업 시작**: `start_jobs()` 로 모든 백그라운드 작업과 서비스 시작
4. **작업 관리자 반환**: 초기화된 `JobManager` 인스턴스 반환

관련 함수들:

- `warn_if_no_services_enabled()` : 설정 유효성 검사
- `load_data_from_database()` : 데이터베이스 데이터 로딩
- `start_jobs()` : 모든 서비스 작업 시작

2.3 부트스트랩 설정 (`bootstrap::app::setup()`)

함수명: `bootstrap::app::setup()`

위치: `src/bootstrap/app.rs`

시그니처:

```rust
#[must_use]
#[instrument(skip())]
pub fn setup() -> (Configuration, AppContainer) {
#[cfg(not(test))]
check_seed();

let configuration = initialize_configuration();

if let Err(e) = configuration.validate() {
panic!("Configuration error: {e}");
}

initialize_global_services(&configuration);

tracing::info!("Configuration:\n{}", configuration.clone().mask_secrets().to_jso

let app_container = AppContainer::initialize(&configuration);

(configuration, app_container)
}
```

역할: 애플리케이션의 기본 설정을 로드하고 전역 서비스를 초기화합니다.

동작 과정:

1. **시드 검증**: 프로덕션 환경에서 적절한 암호화 시드 사용 확인
2. **설정 로딩**: `initialize_configuration()` 으로 환경변수나 파일에서 설정 로드
3. **설정 검증**: `configuration.validate()` 로 설정 유효성 검사
4. **전역 서비스 초기화**: 로깅 시스템과 정적 변수들 초기화
5. **설정 로깅**: 민감한 정보를 마스킹한 설정 정보 로그 출력
6. **컨테이너 초기화**: `AppContainer::initialize()` 로 의존성 주입 컨테이너 생성

관련 함수들:

- `check_seed()` : 암호화 시드 검증
- `initialize_configuration()` : 설정 파일/환경변수 로딩
- `initialize_global_services()` : 전역 서비스 초기화
- `AppContainer::initialize()` : 애플리케이션 컨테이너 초기화

함수명: `initialize_configuration()`

위치: `src/bootstrap/config.rs`

시그니처:

```
#[must_use]
pub fn initialize_configuration() -> Configuration {
let info = Info::new(DEFAULT_PATH_CONFIG.to_string()).expect("info to load conf
Configuration::load(&info).expect("error loading configuration from sources")
}
```

역할: 설정 파일이나 환경변수에서 애플리케이션 설정을 로드합니다.

동작 과정:

1. **기본 경로 설정**: `DEFAULT_PATH_CONFIG` 상수로 기본 설정 파일 경로 지정
2. **설정 정보 생성**: `Info::new()` 로 설정 로딩에 필요한 메타데이터 생성
3. **설정 로딩**: `Configuration::load()` 로 실제 설정 데이터 로드
4. **우선순위**: 환경변수 `TORRUST_TRACKER_CONFIG_TOML` 이 파일보다 우선

설정 소스 우선순위:

1. 환경변수: `TORRUST_TRACKER_CONFIG_TOML`
2. 설정 파일: `./share/default/config/tracker.development.sqlite3.toml`

2.4 컨테이너 초기화 (`AppContainer::initialize()`)

함수명: `AppContainer::initialize()`

위치: `src/container.rs`

시그니처:

```rust
#[instrument(skip(configuration))]
pub fn initialize(configuration: &Configuration) -> AppContainer {
// Configuration
let core_config = Arc::new(configuration.core.clone());
let http_api_config = Arc::new(configuration.http_api.clone());


// Registar
let registar = Arc::new(Registar::default());
```

```
// Swarm Coordination Registry Container
let  swarm_coordination_registry_container = Arc::new(SwarmCoordinationRegistryC
core_config.tracker_usage_statistics.into(),
));


// Core
let  tracker_core_container = Arc::new(TrackerCoreContainer::initialize_from(
&core_config,
&swarm_coordination_registry_container,
));


// HTTP & UDP 컨테이너 초기화...
AppContainer { /* 모든 컴포넌트 */ }
}
```

역할: 의존성 주입 컨테이너를 초기화하여 모든 서비스 컴포넌트들을 생성하고 연결합니다.

동작 과정:

1. **설정 래핑**: 핵심 설정과 HTTP API 설정을 `Arc` 로 래핑하여 스레드 안전하게 공유
2. **레지스트라 초기화**: 서비스 등록 및 관리를 위한 `Registar` 생성
3. **스웜 조정 레지스트리**: 피어 간 조정을 위한 `SwarmCoordinationRegistryContainer` 초기화
4. **트래커 코어**: 핵심 비즈니스 로직을 담당하는 `TrackerCoreContainer` 초기화
5. **HTTP 서비스**: HTTP 트래커 관련 서비스들과 인스턴스 컨테이너들 초기화
6. **UDP 서비스**: UDP 트래커 관련 서비스들과 인스턴스 컨테이너들 초기화

컨테이너 구성 요소:

- **Configuration**: 설정 정보 관리
- **Registar**: 서비스 등록 및 관리
- **SwarmCoordinationRegistryContainer**: 스웜 조정 서비스
- **TrackerCoreContainer**: 핵심 트래커 로직
- **HttpTrackerCoreServices**: HTTP 트래커 서비스들
- **UdpTrackerCoreServices**: UDP 트래커 서비스들
- **Instance Containers**: 각 바인드 주소별 트래커 인스턴스들

2.5 서비스 시작 순서와 의존성 관계

함수명: `start_jobs()`

위치: `src/app.rs`

시그니처:

```
async fn start_jobs(config: &Configuration, app_container: &Arc<AppContainer>)
let mut job_manager = JobManager::new();

// 이벤트 리스너들 시작
start_swarm_coordination_registry_event_listener(config, app_container, &mut jo
start_tracker_core_event_listener(config, app_container, &mut job_manager);
start_http_core_event_listener(config, app_container, &mut job_manager);
start_udp_core_event_listener(config, app_container, &mut job_manager);
start_udp_server_stats_event_listener(config, app_container, &mut job_manager);
```

```
start_udp_server_banning_event_listener(app_container, &mut job_manager);


// 트래커 인스턴스들 시작
start_the_udp_instances(config, app_container, &mut job_manager).await;
start_the_http_instances(config, app_container, &mut job_manager).await;


// 백그라운드 작업들 시작
start_torrent_cleanup(config, app_container, &mut job_manager);
start_peers_inactivity_update(config, app_container, &mut job_manager);


// API 서비스들 시작
start_the_http_api(config, app_container, &mut job_manager).await;
start_health_check_api(config, app_container, &mut job_manager).await;


job_manager
}
```

역할: 모든 서비스와 백그라운드 작업들을 올바른 순서로 시작하고 관리합니다.

서비스 시작 순서:

1. **이벤트 리스너들 (동기적)**

- Swarm Coordination Registry 이벤트 리스너
- Tracker Core 이벤트 리스너
- HTTP Core 이벤트 리스너

- UDP Core 이벤트 리스너
- UDP Server Stats 이벤트 리스너
- UDP Server Banning 이벤트 리스너

2. **트래커 인스턴스들 (비동기적)**

- UDP 트래커 인스턴스들 시작
- HTTP 트래커 인스턴스들 시작

3. **백그라운드 작업들 (동기적)**

- 토렌트 정리 작업 (비활성 피어 제거)
- 피어 비활성 상태 업데이트 작업

4. **API 서비스들 (비동기적)**

- HTTP REST API 서버
- Health Check API 서버

데이터베이스 데이터 로딩 과정

함수명: `load_data_from_database()`

위치: `src/app.rs`

시그니처:

```
async fn load_data_from_database(config: &Configuration, app_container: &Arc<A
load_peer_keys(config, app_container).await;
load_whitelisted_torrents(config, app_container).await;
load_torrent_metrics(config, app_container).await;
```

```
}
```

역할: 애플리케이션 시작 시 데이터베이스에서 필요한 데이터들을 메모리로 로드합니다.

로딩 순서:

1. **피어 키 로딩** (`load_peer_keys()`)

- **조건**: `config.core.private == true` 인 경우에만 실행
- **동작**: 프라이빗 트래커 모드에서 사용할 인증 키들을 데이터베이스에서 로드
- **함수**: `keys_handler.load_peer_keys_from_database().await`

2. **화이트리스트 로딩** (`load_whitelisted_torrents()`)

- **조건**: `config.core.listed == true` 인 경우에만 실행
- **동작**: 허용된 토렌트 목록을 데이터베이스에서 로드
- **함수**: `whitelist_manager.load_whitelist_from_database().await`

3. **토렌트 메트릭 로딩** (`load_torrent_metrics()`)

- **조건**: `config.core.tracker_policy.persistent_torrent_completed_stat == true` 인 경우에만 실행
- **동작**: 지속적인 토렌트 완료 통계를 데이터베이스에서 로드
- **함수**: `bittorrent_tracker_core::statistics::persisted::load_persisted_metrics()`

작업 관리 시스템 (`JobManager`)

함수명: `JobManager::new()`

위치: `src/bootstrap/jobs/manager.rs`

시그니처:

```rust
#[must_use]
pub fn new() -> Self {
Self {
jobs: Vec::new(),
cancellation_token: CancellationToken::new(),
}
}
```

역할: 모든 백그라운드 작업들을 관리하는 중앙 관리자를 생성합니다.

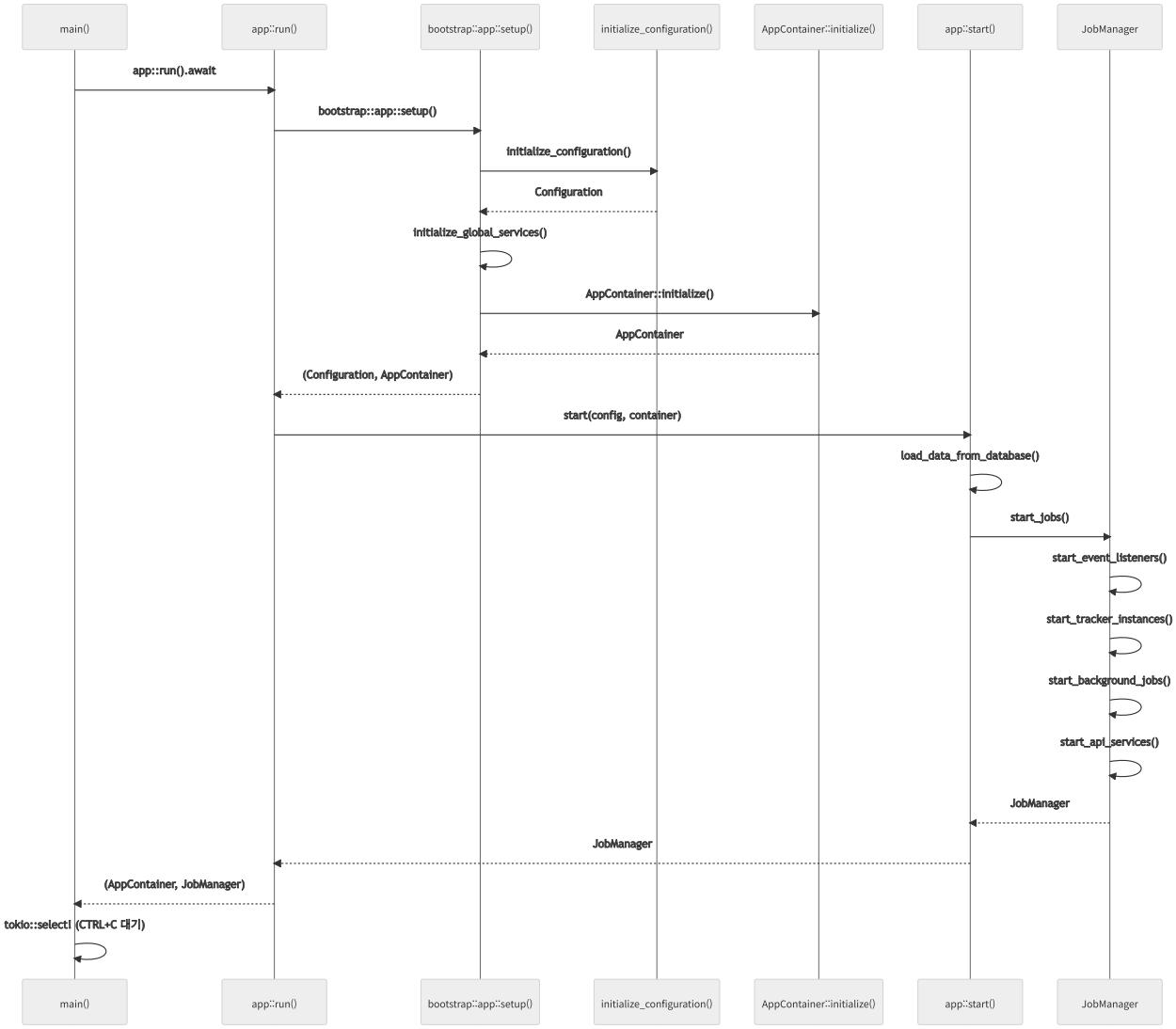
주요 기능:

- **작업 등록**: `push()` 메서드로 새로운 작업 추가
- **취소 토큰**: 모든 작업에 공유되는 취소 신호 관리
- **우아한 종료**: `wait_for_all()` 메서드로 모든 작업의 완료 대기

작업 생명주기:

1. **등록**: `job_manager.push(name, handle)` 로 작업 등록
2. **실행**: 각 작업은 독립적으로 비동기 실행
3. **취소**: `job_manager.cancel()` 로 모든 작업에 취소 신호 전송
4. **대기**: `job_manager.wait_for_all(timeout)` 로 모든 작업 완료 대기

2.6 초기화 과정 다이어그램


main()
app::run()
bootstrap::app::setup()
initialize_configuration()
AppContainer::initialize()
app::start()
JobManager
app::run().await
bootstrap::app::setup()
initialize_configuration()
Configuration
initialize_global_services()
AppContainer::initialize()
AppContainer
(Configuration, AppContainer)
start(config, container)
load_data_from_database()
start_jobs()
start_event_listeners()
start_tracker_instances()
start_background_jobs()
start_api_services()
JobManager
JobManager
(AppContainer, JobManager)
tokio::select! (CTRL+C 대기)
main()
app::run()
bootstrap::app::setup()
initialize_configuration()
AppContainer::initialize()
app::start()
JobManager

2.7 의존성 관계 분석

핵심 의존성 체인

1. **Configuration → AppContainer**

- 설정 정보가 모든 컨테이너 초기화의 기반이 됨
- 각 서비스의 활성화 여부와 설정값 결정

2. **TrackerCoreContainer → Protocol Containers**

- 핵심 비즈니스 로직이 프로토콜별 구현체들의 기반
- HTTP/UDP 트래커 모두 TrackerCore에 의존

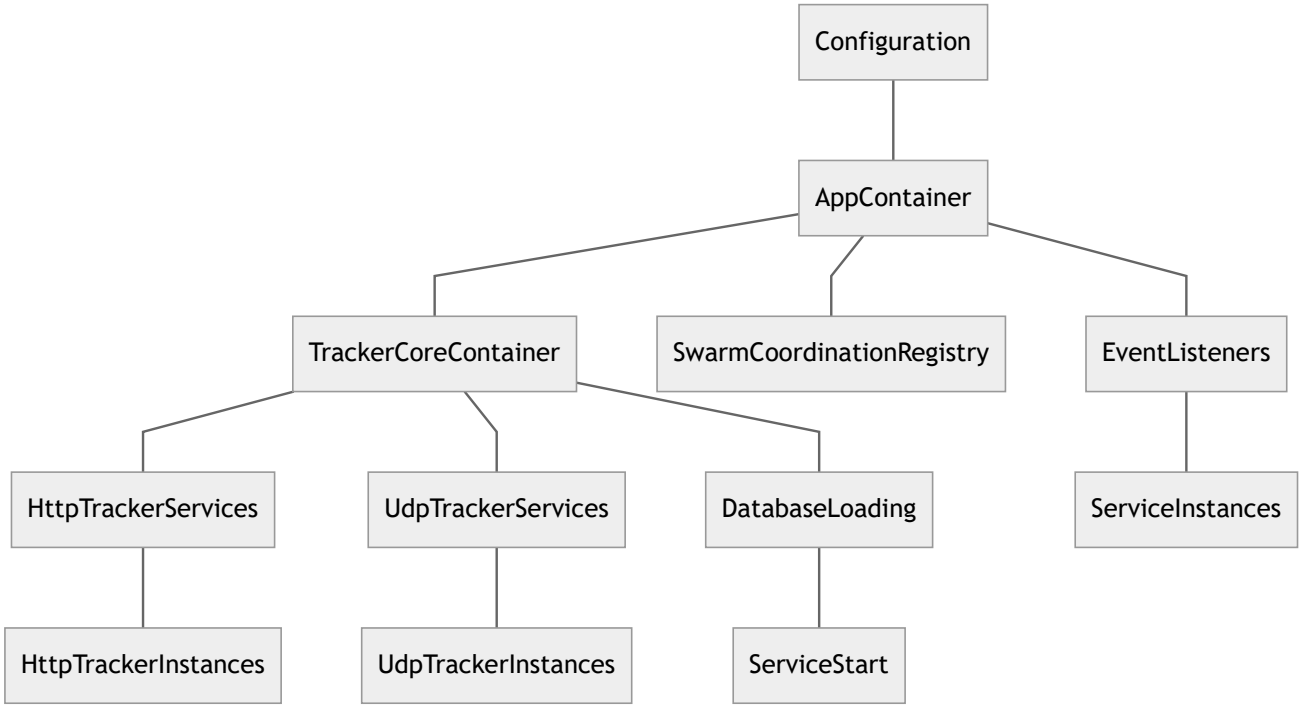
3. **Event Listeners → Service Instances**

- 이벤트 리스너들이 먼저 시작되어야 서비스 이벤트 처리 가능
- 서비스 인스턴스들이 발생시키는 이벤트를 수신

4. **Database Loading → Service Start**

- 데이터베이스 데이터 로딩이 완료된 후 서비스 시작
- 인증 키, 화이트리스트, 메트릭 등의 사전 로딩 필요

서비스 간 의존성



이러한 초기화 과정을 통해 Torrust Tracker는 모든 필요한 컴포넌트들을 올바른 순서로 초기화하고, 안정적인 서비스 운영을 위한 기반을 마련합니다.